Using Knowledge Graphs to Restrict the Context of LLM Analysis: A GraphRAG Approach for Personnel Management

Antonio Gustavo Silveira Dantas[1,*], José Maria Parente de Oliveira[2]

[1]*Núcleo Corporativo de Tecnologia da Informação, São José dos Campos, SP, Brazil*
[2]*Instituto Tecnológico de Aeronáutica, São José dos Campos, SP, Brazil*

Abstract

We investigate the combination of Knowledge Graphs (KGs) and Large Language Models (LLMs) as a means of grounding model responses in verified organizational data and reducing hallucinations. A domain ontology covering people, skills, positions, and projects was defined and instantiated in a Neo4j graph database. A graph-based retrieval-augmented (GraphRAG) pipeline was built on top of this graph: natural language questions are translated into Cypher queries, executed against the KG, and the retrieved structured data is supplied as context for LLM response generation. Three architectures were evaluated on a manually validated gold-standard corpus of 95 question–query–response triples: GPT-4 standalone, a locally-run quantized model fine-tuned for Cypher generation (text-to-cypher-Gemma-3-27B, referred to as HF 27B), and a hybrid combining both. Factual consistency was judged by Claude (model `claude-opus-4-8`), an LLM that is not one of the evaluated systems. HF 27B achieved a perfect 100% Cypher execution rate against 97.9% for GPT-4, but only 49% factual consistency (95% CI 40–59) in the generated answers. GPT-4 reached 70% (95% CI 61–79) and the hybrid 59% (95% CI 49–68). The fully local pipeline trails the fully cloud one by a wide, significant margin (49% vs. 70%, McNemar $p < 0.001$); decomposing this gap shows that the query-generation and answer-generation stages contribute comparably (each isolated step about +10 points, $p \approx 0.06$), so a semantically wrong query and a weak summarizer of verbose results are both liabilities and neither can be skipped. Results support GraphRAG as a viable and practical strategy for domains requiring factual traceability, such as strategic personnel planning and legal research.

Keywords

Knowledge Graphs, Large Language Models, GraphRAG, Hallucination Mitigation, Personnel Management

1. Introduction

The growing adoption of Large Language Models such as GPT-4 [1] and LLaMA [2] across professional settings has brought increased attention to a well-documented failure mode: hallucination. Because these models generate text by sampling from distributions learned during training, they tend to produce fluent but factually unfounded statements whenever a query touches knowledge that was underrepresented in the training corpus or requires precise symbolic reasoning [3, 4]. This is not a fringe concern—surveys have documented hallucination rates severe enough to render LLMs unreliable for high-stakes tasks without additional safeguards [5].

At the same time, Knowledge Graphs (KGs) have become a mature technology for representing domain knowledge in a structured, navigable, and formally grounded form [6]. An ontology provides the conceptual backbone [7], defining which entities exist and what relationships are valid between them; a graph database such as Neo4j [8] makes those structures efficiently queryable. The core hypothesis of this work is that pairing a KG with an LLM—using the graph as the authoritative source of facts and the model as a natural-language interface—can substantially reduce hallucination without sacrificing conversational accessibility [5, 9].

2nd International Workshop on Semantic Technologies for Data Management (ST4DM 2026), July 18, 2026, Lisbon, Portugal

EMAIL: antonio.dantas101835@ga.ita.br (A. G. S. Dantas); parente@ita.br (J. M. P. d. Oliveira)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

Retrieval-Augmented Generation (RAG) [10] provides the architectural pattern for this integration. Standard RAG uses dense-vector similarity search to retrieve relevant documents; graph-based RAG replaces that with symbolic graph queries, which return exactly the facts matching a pattern rather than approximate nearest neighbors. This is particularly advantageous for the kind of multi-hop relational queries that arise in organizational settings—*"Which engineers with Python expertise have worked on cybersecurity projects in the last two years and are eligible for promotion?"*—where vector retrieval tends to fragment and miss [11, 12].

The application domain we chose to study is strategic personnel management, a field with high information complexity and clear costs for errors. Organizations, particularly public ones with large and geographically distributed workforces, routinely need to cross-reference skills, project histories, certifications, and career stages to make staffing and succession decisions. This is precisely the kind of structured, multi-entity query that KGs handle naturally and that LLMs alone handle poorly.

This paper makes the following contributions:

1. A purpose-built domain ontology and Neo4j Knowledge Graph for organizational personnel and knowledge management, together with an explicit account of its design methodology (competency questions) and a discussion of vocabulary reuse and the property-graph-vs-RDF design decision (Section 3.1).
2. A GraphRAG pipeline evaluated under three LLM configurations: GPT-4 standalone, a Cypher-specialized open-source model (HF 27B), and a hybrid combining both, compared on a manually validated 95-question benchmark with confidence intervals and paired significance testing.
3. A structured, data-driven failure analysis that separates syntactic query errors, semantic query errors, and answer-generation (summarization) errors, quantifying how often each occurs and which one binds overall performance.
4. A discussion of deployment trade-offs—cost, latency, data sovereignty, and accuracy—that extends the results to practical organizational decisions.

2. Background and Related Work

2.1. Large Language Models and the Hallucination Problem

LLMs are neural networks based on the Transformer architecture [13] trained on large text corpora to model conditional token distributions [14]. At inference time, they autoregressively generate text by sampling the most probable next token given a context window. This mechanism makes them powerful generalists but also fundamentally probabilistic: they do not retrieve stored facts but reconstruct plausible continuations [3]. When training data is sparse or inconsistent for a given topic, the model may produce confident fabrications. Injecting verified facts as context at inference time is the main mitigation strategy [12, 5].

2.2. Knowledge Graphs and Ontologies

A graph $G(V, E)$ is a set of vertices V and edges E connecting pairs of vertices [15]. In a *labeled property graph*—the model used by Neo4j—vertices and edges carry type labels and key-value attributes, enabling rich, typed relational representations. A Knowledge Graph combines a graph with a formal ontology, an explicit specification of a conceptualization [7] that defines the entity types and admissible relationships of a domain [6]. The ontology provides the schema against which instances are stored and queried.

Two modelling traditions coexist. The Semantic Web stack (RDF/OWL, queried with SPARQL) encodes the ontology itself as data and supports formal reasoning and standardized constraint languages such as SHACL. Labeled property graphs prioritize traversal performance

and developer ergonomics but, unlike RDF/OWL, have *no standardized way of encoding an ontology as part of the graph* [6]: the schema is typically enforced by the application layer rather than by the data model. We return to this trade-off, and to our reasons for adopting a property graph, in Section 3.1. It is also worth tempering a common contrast: relational databases do support schema changes (`ALTER`) and cross-table navigation (`JOIN`); the advantage of a property graph is not that these are impossible elsewhere but that it *facilitates* schema evolution and makes multi-hop traversal a native, first-class operation rather than a chain of joins.

2.3. Retrieval-Augmented Generation and GraphRAG

RAG, introduced by Lewis et al. [10], augments LLM generation with an external retrieval step: relevant documents are fetched and prepended to the prompt so that the model answers from evidence rather than from parametric memory alone. Standard implementations retrieve by dense vector similarity [12], which works well for localized queries but breaks down on global ones—queries that require synthesizing information spread across many documents or entity chains [11].

Graph-based RAG replaces or augments vector retrieval with structured graph access. In the literature the term covers a spectrum. At one end, systems such as the GraphRAG of Edge et al. [11] build a graph from *unstructured* text using an LLM and then traverse community summaries to answer query-focused-summarization questions. At the other end—the setting of this paper—the graph already exists as curated structured data, and retrieval is performed by translating the user's question into a formal graph query (here, Cypher) whose result set is the evidence. Our pipeline therefore belongs to the text-to-query / KG question-answering family: it is "GraphRAG" in the sense that generation is augmented with facts retrieved from a graph, but it does not perform the multi-hop neighborhood exploration or LLM-driven graph construction associated with the unstructured variant. We make this scope explicit because the two families have different failure modes; ours concentrates the risk in the question-to-query translation step. The LangChain library [16] provides infrastructure for this pattern through its `GraphCypherQAChain` abstraction.

2.4. Context Selection and Related Approaches

Even when the graph is the right source, supplying it well matters: irrelevant graph neighborhoods inflate the prompt and degrade response quality. Several lines of work inform our design. KGPool [17] performs dynamic context selection, conditioning graph expansion on the input sentence; the lesson we adopt is that context should be *conditioned on the question* rather than fixed—in our pipeline the Cypher query plays exactly this role, selecting the subgraph relevant to the question instead of dumping a neighborhood. KG-CF [18] and the retrieve-rank-reason loop of Xu et al. [19] show that *pruning* uninformative paths improves downstream accuracy; this motivates our choice to restrict context to the precise result set returned by the query rather than the full neighborhood of the matched entities. At the level of integration patterns, Pan et al. [9] map the landscape of LLM–KG synergy—LLMs assisting KG construction and KGs grounding LLM inference—and our system instantiates the second direction. The chief difference between these works and ours is one of task: they target relation extraction or KG completion, whereas we use the KG as a fixed, authoritative store and study the reliability of the natural-language interface over it.

A practical concern for evaluation is benchmark quality. Zhang et al. [20] identified annotation artifacts in widely used KG-RAG datasets that inflate reported accuracy and mask real failure modes. Their findings motivated the manual validation procedure we describe in Section 3.3.

3. Methodology

3.1. Domain Ontology and Knowledge Graph Construction

Design methodology. The ontology was developed following a lightweight, competency-question-driven process in the spirit of established ontology-engineering methodologies. We first elicited the information needs of strategic personnel management as a set of *competency questions*—natural-language questions the system should be able to answer, such as "which people of a given rank hold a given skill?", "which projects require a given technology?", and "which knowledge domains are held by fewer than k people?". Each candidate class and relationship was admitted only if it was required to answer at least one competency question that could not otherwise be expressed precisely [7]. We emphasize that these competency questions guided the *schema*; they are conceptually distinct from the evaluation benchmark of Section 3.3, although both draw on the same domain.

Vocabulary reuse. For a domain centered on people and organizations, established vocabularies such as `foaf:Person` or `schema.org/Person` are natural reuse candidates, and in an RDF setting we would align to them. We deliberately did not import them here for two reasons. First, the deployment target processes sensitive personnel records under data-sovereignty constraints and required Portuguese, domain-specific labels (e.g., `patentePessoa` for military rank) with no direct counterpart in general vocabularies. Second, the property-graph model we adopted has no standardized OWL-style import mechanism, so reuse would be nominal rather than semantic. We regard alignment to `foaf`/`schema.org` (and publication of the ontology in OWL) as worthwhile future work for interoperability, and we note it as a limitation in Section 5.

Why a property graph. We selected Neo4j and a labeled property graph—rather than RDF/SPARQL/OWL—for pragmatic engineering reasons: native multi-hop traversal performance, mature tooling and drivers for the local, on-premises deployment our use case demands, and direct support from the `GraphCypherQAChain` translation tooling. The cost of this choice, acknowledged here, is reduced access to formal reasoning and to standardized validation (e.g., SHACL); Section 4.3 shows that a schema/ontology-aware validation step is precisely what the dominant error class would benefit from, which makes the trade-off a substantive one rather than a neutral implementation detail.

Resulting schema. The schema defines fourteen node labels: `Pessoa` (Person), `Competencia` (Skill), `Projeto` (Project), `Conhecimento` (Knowledge), `Tecnologia` (Technology), `Treinamento` (Training), `Certificacao` (Certification), `Documentacao` (Documentation), `Processo` (Process), `Pesquisa` (Research), `Publicacao` (Publication), `Graduacao` (undergraduate degree), `PosGraduacao` (graduate degree), and `Sumario` (abstract). Attributes such as a person's rank and position are stored as `Pessoa` properties (`patentePessoa`, `cargoPessoa`) rather than as separate nodes. These labels are connected by eighteen typed, directed relationship types such as `TEM_COMPETENCIA` (has skill), `TRABALHA_EM` (works in), `UTILIZA_TECNOLOGIA` (uses technology), `CONCLUIU_TREINAMENTO` (completed training), and `POSSUI_CERTIFICACAO` (holds certification). The schema is shown in Figure 1. The graph was implemented in Neo4j [8] and populated with synthetic but representative data using Python scripts (Section 3.3 discusses the synthetic data and its limitations). The populated graph contains 145 nodes (70 `Pessoa`, the rest distributed over the remaining labels) and 1,342 relationships across the eighteen types, the most frequent being `INDICADO_PARA` and `CONCLUIU_TREINAMENTO` (215 each), `TRABALHA_EM` (196), `TEM_COMPETENCIA` (169), and `POSSUI_CONHECIMENTO` (158).

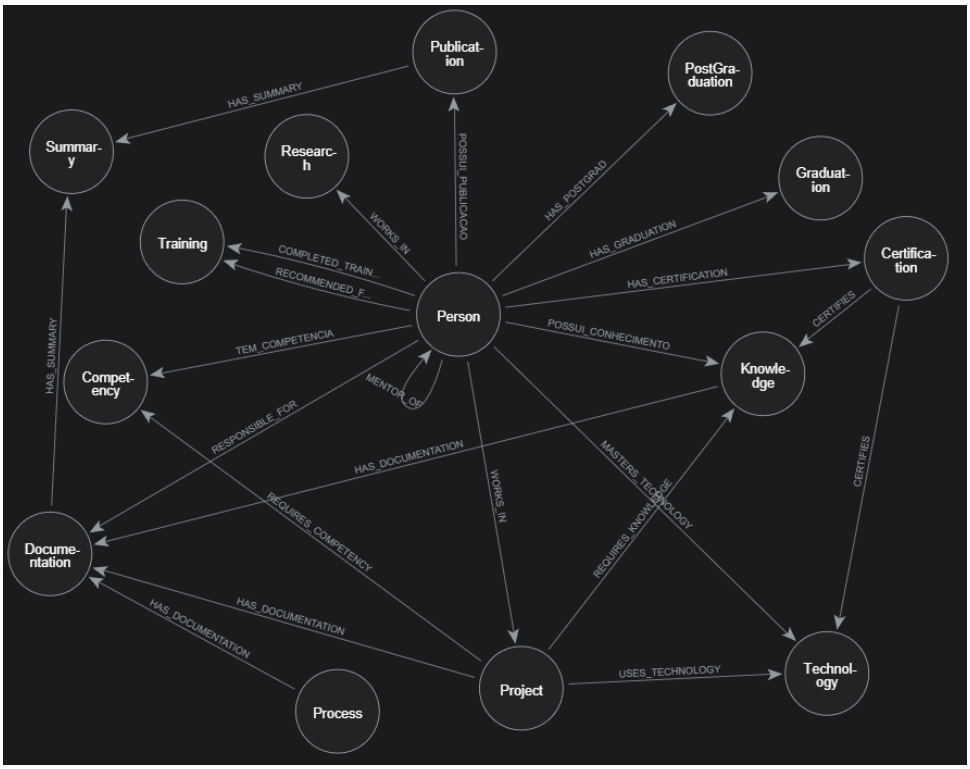


Figure 1: The Knowledge Graph schema used in this study. Nodes represent domain entity types; edges represent typed relationships.

3.2. GraphRAG Pipeline

The pipeline has four stages, illustrated in Figure 2: (i) a *translation* module converts the natural language question into Cypher using an LLM; (ii) a *retrieval* module executes that Cypher against the KG and returns a precise result set; (iii) an *augmentation* module serializes the result set as structured (JSON) context; and (iv) a *generation* module produces a grounded natural language answer. Compared to vector RAG, this eliminates retrieval approximation—if the Cypher is correct, the facts are exact—but shifts the principal failure mode to query semantic correctness. It was implemented in Python using Neo4j's official driver and LangChain [16] for prompt management.

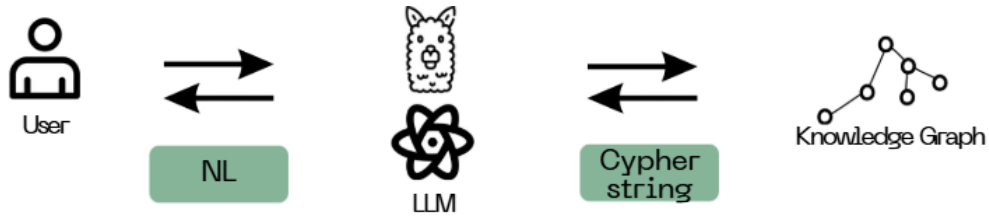


Figure 2: The GraphRAG pipeline. The user submits a natural language question; an LLM translates it into Cypher; Neo4j executes the query and returns structured data; a second LLM step generates the final answer grounded in that data.

Prompts and control flow. For reproducibility we make the prompt design explicit. The translation prompt supplies the LLM with the full graph schema (node labels, relationship types, and property keys) and the user question, and instructs it to return a single Cypher statement. The generation prompt receives *both the original natural-language question and the JSON result set*, and is instructed to answer the question using only the supplied data, without recourse to

parametric knowledge. When the query returns an empty result set, the generator is instructed to emit a fixed phrase ("I did not find information about this in the database") rather than speculate.

Configurations. Three configurations were evaluated to disentangle the contributions of query generation and answer generation:

- **GPT-4 standalone.** GPT-4 [1] handles both the NL-to-Cypher translation and the final answer generation via the OpenAI API. This represents the cloud-based LLM-as-a-service paradigm and serves as the primary accuracy baseline.
- **HF 27B standalone.** Both stages use the Hugging Face `text-to-cypher-Gemma-3-27B-Instruct` model, quantized to 4-bit precision (Q4) and served locally via Ollama. It was selected because it runs entirely on local hardware—a key constraint for organizations processing sensitive personnel data—and because it is specifically fine-tuned for Cypher generation, enabling a comparison between domain specialization and general capability.
- **Hybrid.** HF 27B generates the Cypher query; GPT-4 summarizes the JSON result into natural language. Because the answer generator is held fixed (GPT-4) and only the upstream query source changes, the gap between the hybrid and GPT-4 standalone isolates the cost of the query-generation stage.

Local inference ran on a consumer workstation with an NVIDIA GeForce RTX 5060 Ti (16 GB GDDR6), AMD Ryzen 5 9600X, and 32 GB DDR5 RAM.

3.3. Evaluation Dataset and Metrics

Taking the critique of Zhang et al. [20] seriously, we applied a hybrid but primarily manual construction procedure. An auxiliary LLM first generated a pool of candidate questions by systematically varying entity types and relationship combinations present in the ontology. Every candidate was then revised by the authors: questions were rewritten for clarity and appropriate complexity; gold-standard Cypher queries were written and tested directly in Neo4j rather than being LLM-generated; and reference answers were composed manually to ensure factual correctness against the graph. The final corpus contains 95 question–query–response triples.

Query-complexity breakdown. To characterize the benchmark—and to enable a per-class analysis (Section 4.3)—we classify each item by the structure of its gold Cypher query into four classes: *property lookups* (single node, no traversal; 18 items), *single-hop* relational queries (one relationship pattern; 70 items), *multi-pattern* conjunctive queries (two or more patterns, e.g., "uses both Python and PostgreSQL"; 6 items), and *aggregation* queries (1 item). This distribution is itself informative: the benchmark is dominated by single-hop lookups, which we flag as a representativeness limitation in Section 5. We also note that the questions, while reviewed by the authors, were not validated by an external domain expert, and that establishing their representativeness of real user queries is future work.

Metrics. Four metrics were computed for each configuration:

- **Cypher Execution Rate.** The fraction of generated queries that Neo4j executes without error. This captures syntactic validity and *schema compatibility at runtime*—i.e., that the query references only node labels, relationship types, and property keys that exist in the graph. It does *not* certify that the query is semantically correct.
- **Factual Consistency (LLM-as-a-Judge).** Each generated response was rated *Consistent*, *Partially Consistent*, or *Inconsistent* against the gold-standard answer, on a structured rubric, by an LLM judge. We use **Claude Opus 4 (Anthropic, model**

`claude-opus-4-8`). Crucially, the judge is *not* one of the three evaluated configurations, which removes the self-preference risk that arises when a system under test also judges. An answer is rated *Consistent* when it conveys the gold answer's entities and facts without contradiction (extra grounded detail is allowed), *Partially Consistent* when correct but incomplete, and *Inconsistent* otherwise (wrong entities, fabrication, or a "not found" reply when data exist). This is our primary accuracy signal; a comparison against a GPT-4 judge is reported in Section 4.3.

- **BERTScore F1** [21]. Token-level semantic similarity between the generated response and the gold standard, used as a secondary linguistic-quality indicator.
- **Cost and latency.** Wall-clock evaluation time and monetary cost per configuration (Section 4.1).

Statistical reporting. With $n = 95$ we report Consistent rates as rounded percentages with Wilson 95% confidence intervals [22], avoiding two-decimal rates that would overstate precision at this sample size. Paired configurations are compared on categorical consistency with McNemar's exact test [23] and on BERTScore with the Kruskal–Wallis and Wilcoxon signed-rank tests; agreement between the two LLM judges is quantified with Cohen's κ [24]. The script and labelled data that reproduce every reported figure are released with the paper (Section 5).

4. Results and Discussion

4.1. Infrastructure and Cost Trade-offs

Before turning to accuracy, Table 1 records the practical costs of each configuration, as these shape real deployment decisions. Evaluating the full 95-question dataset took approximately 25 minutes with GPT-4, at roughly US$ 7.50 in API fees. HF 27B required around five hours on the available hardware; its marginal monetary cost was negligible, as both inference and consistency judging were performed offline. The hybrid configuration combines a local HF 27B translation pass (part of the same five-hour local budget) with a GPT-4 summarization pass of about 10 minutes and US$ 4.70.

Table 1
Cost and latency over the 95-question benchmark. Local time for the hybrid refers to the HF 27B translation pass; cloud time refers to the GPT-4 summarization pass.

Configuration	Wall-clock time	Hardware	Monetary cost
GPT-4 standalone	≈25 min	cloud API	≈US$ 7.50
HF 27B standalone	≈5 h	local GPU	≈US$ 0*
Hybrid	≈5 h (local) + 10 min (cloud)	local GPU + cloud API	≈US$ 4.70

*Local inference only; near-zero marginal monetary cost (consistency judging done offline).

These figures illustrate a trade-off that goes beyond accuracy: cloud-based APIs offer low latency and high performance with no infrastructure investment, but introduce recurring costs, vendor dependency, and potential data exposure; local models invert this profile, offering data sovereignty and near-zero marginal cost at the expense of hardware investment and significantly higher inference time. For organizations handling sensitive personnel records, the local option may be preferable even at a performance penalty.

4.2. Cypher Query Generation

Table 2 summarizes the quantitative results. The HF 27B model, fine-tuned specifically for text-to-Cypher translation, achieved a perfect 100% execution rate; GPT-4 reached 97.9%

(93/95). Manual inspection of GPT-4's two failures found that both originated from surface-level string formatting issues—mismatched quote characters and missing label backticks—reported uniformly by Neo4j as `Neo.ClientError.Statement.SyntaxError`, rather than from any misunderstanding of the question. GPT-4 understood *what* to retrieve but occasionally stumbled on exact syntax, whereas HF 27B's fine-tuning made it essentially immune to these surface errors.

We note a low-cost remedy that would likely close this small gap entirely: feeding the Neo4j syntax error back to GPT-4 with a request to repair the query. Because this would be triggered in only 2.1% of cases, the overhead is negligible; we did not apply it here in order to keep the three configurations directly comparable, but it is the obvious first step for a production deployment and would reduce the text-to-Cypher choice to "cloud vs. fully local" rather than requiring a specialized model for syntactic robustness alone.

Table 2
Comparative performance of the three architectures on the 95-question gold-standard benchmark. The consistency categories and the error category together partition all 95 items in each column (the underlying counts sum to 95; rounded percentages may differ from 100 by up to one point). Factual consistency is judged by Claude Opus 4 (`claude-opus-4-8`); bracketed values for the *Consistent* row are Wilson 95% confidence intervals.

Metric	GPT-4	HF 27B	Hybrid (HF 27B + GPT-4)
Cypher Execution Rate	97.9%	100%	100%
Query-level entity recall (vs. gold)	63%	68%	68%
BERTScore F1 (mean)	0.793	0.760	0.742
Factual Consistency: Consistent	70% [61–79]	49% [40–59]	59% [49–68]
Factual Consistency: Partial	6%	6%	9%
Factual Consistency: Inconsistent	21%	44%	32%
No answer (API/exec. error)[†]	2%	0%	0%

[†]Two GPT-4 items produced no answer because the generation call hit an API rate limit; with no text to assess they are counted as not consistent and listed here for transparency. With Claude as judge every item that has an answer was scored, so there are no judge-scoring errors (the earlier GPT-4 judge had left 6 HF 27B and 2 hybrid items unscored).

4.3. Factual Consistency of Generated Answers

The accuracy picture reverses when we look at response quality. Judged by Claude (Section 3.3), GPT-4 standalone produced consistent answers 70% of the time (95% CI 61–79), the highest of the three; HF 27B reached 49% (95% CI 40–59) and the hybrid 59% (95% CI 49–68).

Both pipeline stages contribute to the gap. Two paired comparisons localize where quality is lost, both scored by the same judge. Holding the *query* fixed and changing only the generator (HF 27B standalone → hybrid) raised consistency from 49% to 59%, isolating the value of a stronger summarizer (McNemar exact test, $p = 0.06$). Holding the *generator* fixed at GPT-4 and changing only the query source (hybrid → GPT-4 standalone) raised consistency from 59% to 70%, isolating the cost of HF 27B's retrieval (McNemar, $p = 0.06$). The two stages thus contribute comparably, and at $n = 95$ each individual gap is only marginally significant; what is unambiguous is the end-to-end difference between the fully local and the fully cloud pipeline (HF 27B 49% vs. GPT-4 70%, $p < 0.001$). Neither stage can be skipped: a semantically wrong query cannot be repaired by the generator, and verbose whole-node retrieval defeats a weak summarizer. Because the judge (Claude) is not one of the evaluated systems, none of these comparisons is affected by judge self-preference. We examine the two failure routes next.

A direct query-level check. Because gold Cypher exists for all 95 items, we executed every generated query against the populated graph and compared its result with the gold query's.

Exact result-set identity is low and—misleadingly—similar across configurations (GPT-4 14%, HF 27B/hybrid 18%): it is dominated by *projection* differences (selecting different properties, or returning a whole node where the gold projects two fields), not by retrieving the wrong entities. A projection-robust comparison that matches on the set of returned entity *names* gives a fairer picture: the generated query recovers all gold entities in 63% of items for GPT-4 and 68% for HF 27B (the hybrid inherits HF 27B's queries, hence 68%), with exact name-set agreement of 61% and 58%, respectively. The key observation is that HF 27B's queries are *at least as accurate as GPT-4's at the entity level*—consistent with only 7/95 targeting the wrong labels—so HF 27B's far lower end-to-end consistency (49%) is not caused by worse retrieval but by the answer-generation step. This is the clearest evidence that the local model's binding weakness is the summarization of its verbose, whole-node results rather than query semantics, and it is why our primary accuracy signal is the Claude-judged answer consistency rather than a raw result-set match.

What actually goes wrong upstream. A direct inspection of HF 27B's 95 generated queries refines the usual "the model writes semantically wrong Cypher" narrative. Genuine schema-element errors—where the query targets a different node label than the gold query—occur in only **7 of 95** cases. A representative example is question Q38, "which people are data scientists and master Neo4j?": the gold query treats "data scientist" as a `Cargo` (position) attribute, whereas HF 27B modelled it as a `Competencia` (skill) and returned nothing (Listing 1). This is the failure mode that matters most—it executes cleanly and is invisible to the execution-rate metric—but it is not the most frequent one.

Listing 1: A genuine semantic query error (Q38). The gold query filters on the `Cargo` "Cientista de Dados"; HF 27B instead looks for a `Competencia` containing "ciência de dados", which does not exist, so the executable query returns an empty set.

```
-- Gold (correct schema element: Cargo)
MATCH (p:Pessoa {cargoPessoa:'Cientista de Dados'})
      -[:DOMINA_TECNOLOGIA]->(t:Tecnologia {nomeTecnologia:'Neo4j'})
RETURN p.idPessoa, p.nomePessoa

-- HF 27B (wrong schema element: Competencia)
MATCH (p:Pessoa)-[:TEM_COMPETENCIA]->(c:Competencia)
WHERE toLower(c.nomeCompetencia) CONTAINS 'ciência de dados'
  AND (p)-[:DOMINA_TECNOLOGIA]->(:Tecnologia {nomeTecnologia:'Neo4j'})
RETURN p
```

More frequently, the query is schema-correct but the *answer generation* fails. HF 27B returned the whole matched node (`RETURN p`) in 47 of 95 queries—versus 10 of 95 for GPT-4, which tends to project specific properties (`RETURN p.nomePessoa`). Returning whole nodes produces a verbose, deeply nested JSON payload that the summarization step handles poorly. Of HF 27B's 51 "I did not find information" answers, **46 were produced over a schema-correct query**; the empties are therefore predominantly summarization failures, not retrieval failures. The decisive evidence is what happens when the *same* retrieval is summarized by GPT-4 instead: in 11 of those 46 cases the hybrid recovered a correct, non-empty answer. Question Q1 is the canonical example (Listing 2): given identical Cypher and identical results, HF 27B reported "not found" while GPT-4 correctly named the two people.

Listing 2: A summarization failure (Q1). HF 27B and the hybrid execute the *same* query; HF 27B's own generator reports an empty answer, whereas GPT-4 summarizing the identical result set returns the correct people. This isolates answer generation, not retrieval, as the fault.

```
-- Query generated by HF 27B (used by both HF 27B standalone and Hybrid)
MATCH (p:Pessoa)-[:TEM_COMPETENCIA]->(c:Competencia)
WHERE toLower(p.patentePessoa) CONTAINS 'major'
  AND toLower(c.nomeCompetencia) CONTAINS 'gestão de projetos'
RETURN p
```

```
-- HF 27B answer:  "I did not find information about this in the database."
-- Hybrid answer (GPT-4 over the same results): "Militar 62 and Militar 65 ..."
```

These three patterns—genuine schema-element errors (rare but consequential), whole-node retrieval that overwhelms summarization, and prompt-instruction violations (e.g., speculating instead of emitting the standardized empty-result phrase)—are consistent with HF 27B being tuned narrowly for text-to-Cypher *translation* rather than for the distinct task of summarizing structured results. A different locally-run model dedicated to the generation step, or a generation-aware fine-tune, is a natural remedy.

An ontology-aware validation step would help. The most consequential error—an executable query that retrieves the wrong data—is exactly what schema- or ontology-aware validation could catch. Rather than checking only that the query executes, the system could verify that it uses the intended classes and relationships and that its result shape matches the question's expected interpretation, via constrained/schema-guided generation, query templates, or SHACL-style checks on the result set before answer generation. This is the main semantic lever left unexploited by the current property-graph implementation and a primary direction for future work.

Per-class breakdown. Table 3 reports consistency by query class, scored by the same Claude judge used throughout, so it is directly comparable with Table 2. The GPT-4 advantage is concentrated in single-hop queries (73% vs. 53% for the hybrid and 44% for HF 27B), where HF 27B's whole-node retrieval most often derails summarization; property lookups, by contrast, are handled comparably by all three, and the small multi-pattern set is essentially even.

Table 3
Consistent-answer rate by query class, scored by the same Claude judge used throughout, directly comparable with Table 2. The benchmark is dominated by single-hop queries, where the upstream-query effect is largest.

Query class	n	GPT-4	HF 27B	Hybrid
Property lookup	18	56%	61%	67%
Single-hop	70	73%	44%	53%
Multi-pattern	6	83%	83%	100%
Aggregation	1	100%	0%	100%

Judge reliability. Using an LLM as judge is defensible here for two reasons. First, our judge (Claude) is not among the three evaluated configurations, so the self-preference bias that arises when a system under test also judges is structurally absent. Second, to check that the judgments are not idiosyncratic, we re-scored the items with a second, independent LLM judge (GPT-4) and found high agreement with Claude—98% exact agreement (Cohen's $\kappa = 0.95$) on the GPT-4 configuration and 94% ($\kappa = 0.89$) on the hybrid, the two configurations for which per-item GPT-4-judge labels are available. The two judges agree on the ranking and on the wide GPT-4-vs-HF 27B gap ($p < 0.001$ under both); they differ mainly in leniency toward verbose-but-grounded answers, which slightly shifts absolute rates (the GPT-4 judge scored the hybrid at 53% vs. Claude's 59%). We adopt Claude as the primary judge because it is bias-free by construction and because the GPT-4-judge run left eight items unscored due to API rate-limiting. A human-annotated subsample remains the most desirable next step.

4.4. BERTScore Analysis

BERTScore [21] compares the contextual token embeddings of the generated and reference texts, rewarding semantic equivalence even when surface words differ (e.g., "senior engineer" vs.

"level-3 developer"). It is well suited to crediting valid paraphrases of a fact, but—because it operates on token-embedding similarity—it does *not* verify entity identity or numeric precision: an answer naming the wrong person in the right vocabulary still scores fairly high. We therefore treat factual consistency as primary and BERTScore as a secondary, complementary signal.

Figure 3 shows the BERTScore F1 distribution for the three configurations as box plots, which summarize the median, quartiles, and spread far more legibly than the per-question scatter plots of the previous version. GPT-4 has the highest median (0.82) and the two near-zero outliers correspond exactly to its two execution failures. HF 27B (mean 0.760) is more dispersed with a longer lower tail, reflecting summarization over malformed or empty context; the hybrid (mean 0.742) has the lowest median. A Kruskal–Wallis test confirms the distributions differ ($H = 10.8$, $p = 0.004$); pairwise Wilcoxon signed-rank tests show GPT-4 significantly above both HF 27B ($p = 0.002$) and the hybrid ($p < 0.001$), while HF 27B vs. hybrid is *not* significant ($p = 0.08$).

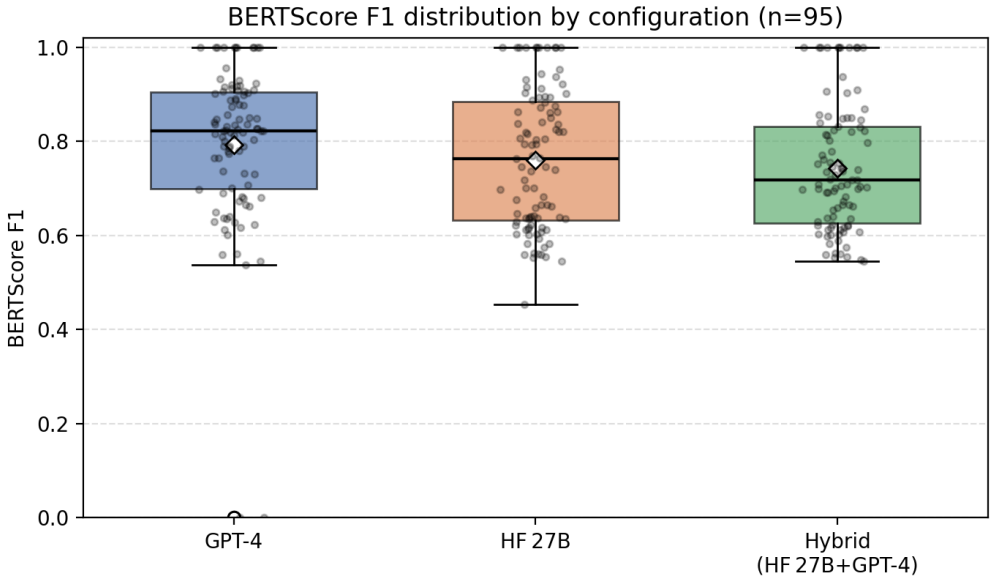


Figure 3: BERTScore F1 distribution per configuration ($n = 95$). Boxes show the interquartile range and median; diamonds mark the mean; points are individual questions (jittered). GPT-4's two near-zero points are its execution failures.

The counter-intuitive result—that the hybrid scores no higher than HF 27B on BERTScore despite using the stronger GPT-4 generator—has a concrete explanation that the non-significant difference already tempers. Because HF 27B returns whole nodes, GPT-4 in the hybrid is handed verbose JSON and tends to produce longer, detail-padded answers (e.g., in Q1 the hybrid appended each person's position, admission date, and e-mail, earning a *Partially Consistent* rating). These additions are factually grounded but diverge in surface form from the concise, manually written gold answers, which depresses token-level similarity. This is an interaction between retrieval shape and generation, not evidence that GPT-4 is a weaker summarizer; it also illustrates why we do not lean on BERTScore as a primary metric for a factual-traceability setting.

4.5. Application Scenarios

The benchmark queries reflect genuine organizational information needs. We highlight four areas where the architecture is immediately actionable. **Strategic workforce planning**: queries such as *"list all technologies required by Project X for which we have fewer than three practitioners at senior level"* expose skill gaps for proactive hiring and training. **Knowledge succession**: *"which knowledge domains critical to Project Y are held by fewer than two people?"*

surfaces institutional fragility before it becomes a crisis—directly relevant in organizations with high turnover or mandatory rotation, such as the military context that motivated this study. **Dynamic personnel allocation**: *"find all active specialists holding certification Z who completed training W in the last 24 months and are not on a critical project"* enables rapid, auditable staffing decisions—the answer is a traversal result, not an LLM opinion. **Legal and regulatory research**: a KG of legislation, decisions, and citations would let compliance officers ask provenance questions whose answers are grounded in graph structure, where hallucination carries direct legal consequence.

5. Limitations

Several limitations bound our conclusions. **Synthetic data and benchmark scope**: the KG was populated with synthetic data and the 95-item benchmark, while manually validated, was seeded by an LLM and not reviewed by an external domain expert; it is also skewed toward single-hop queries (Section 3.3), so the results may not transfer to richer multi-hop or aggregation workloads. A real organizational case study is needed. **Statistical power**: with $n = 95$, confidence intervals are wide and some differences (e.g., HF 27B vs. hybrid on BERTScore) are not significant; absolute rates should be read with their intervals. **Judge**: factual consistency is assessed by an LLM judge (Claude, `claude-opus-4-8`) deliberately not among the evaluated systems; a second LLM judge (GPT-4) agrees closely (98%/94%, $\kappa \geq 0.89$), but no human annotation was performed, which remains desirable. **Single-run estimates**: each configuration was evaluated once, without multiple seeds or temperature settings, so run-to-run variance is not quantified. **Unexploited semantics**: the KG is used as a retrieval store without reasoning, constraints, or validation, and the ontology is not yet published in OWL or aligned to standard vocabularies.

Reproducibility. To address reproducibility, the domain ontology, the synthetic-data generation scripts, the Neo4j dump, the 95-item benchmark, the pipeline prompts, the labelled per-item evaluation data, and the analysis script (`analysis.py`) that recomputes every reported statistic are made available at https://github.com/INSERT-REPOSITORY-URL.[1]

6. Conclusion

This paper evaluated a GraphRAG architecture for strategic personnel management across three LLM configurations and a 95-question gold-standard benchmark. The central finding is a clear division of labor: a domain-specialized model excels at syntactic Cypher generation (100% execution for HF 27B), while a general-purpose proprietary model leads on factual consistency of the generated answer (70% for GPT-4, judged by Claude). Decomposing the local-vs-cloud gap with a single fixed judge shows that the query-generation and answer-generation stages contribute comparably (each isolated step raises consistency by about ten points, each only marginal at $n = 95$), while the end-to-end gap is large and robust (49% vs. 70%, $p < 0.001$). A finer-grained analysis shows that HF 27B's deficit is driven less by frankly wrong queries (7/95) than by schema-correct queries whose whole-node result shape defeats summarization—a distinction with direct design implications.

Beyond accuracy, the comparison surfaces a cloud-versus-local trade-off relevant to real organizations: GPT-4 offers the highest accuracy with minimal infrastructure but recurring cost and external data exposure; HF 27B offers data sovereignty and near-zero marginal cost but currently lags on end-to-end answer quality. The path forward is concrete: (i) add an ontology-aware query-validation step (constrained generation, templates, or SHACL-style result

[1]Repository URL to be inserted for the camera-ready version.

checks) to catch the executable-but-wrong queries; (ii) fine-tune an open model end-to-end on full pipeline trajectories (question → Cypher → JSON → answer) and project specific properties rather than whole nodes; (iii) evaluate larger, unquantized models on server-class hardware to separate quantization from capacity; and (iv) build larger, expert-validated KG-RAG benchmarks following the criteria of Zhang et al. [20] to enable statistically robust comparison.

Acknowledgments

We thank the Instituto Tecnológico de Aeronáutica (ITA) for institutional support. We also thank Major Aviador Douglas Silva Teixeira (CCA-SJ) for co-advising the undergraduate thesis from which this paper originates, and the open-source communities behind Neo4j, LangChain, and the Hugging Face ecosystem for making this research reproducible.

Declaration on Generative AI

During the preparation of this work the authors used Google Gemini to assist with English translation and LaTeX formatting. All content was subsequently reviewed and edited by the authors, who take full responsibility for the final publication.

References

[1] OpenAI, GPT-4 Technical Report, Technical Report arXiv:2303.08774, OpenAI, 2023. URL: https://arxiv.org/abs/2303.08774. doi:10.48550/arXiv.2303.08774.

[2] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, Llama: Open and efficient foundation language models, arXiv preprint arXiv:2302.13971 (2023).

[3] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, H. Hajishirzi, When not to trust language models: Investigating effectiveness of parametric and non-parametric memories, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Toronto, Canada, 2023, pp. 9802–9822. URL: https://aclanthology.org/2023.acl-long.546/. doi:10.18653/v1/2023.acl-long.546.

[4] J. Huang, K. C.-C. Chang, Towards reasoning in large language models: A survey, in: Findings of the Association for Computational Linguistics: ACL 2023, Association for Computational Linguistics, Toronto, Canada, 2023, pp. 1049–1065. URL: https://aclanthology.org/2023.findings-acl.67/. doi:10.18653/v1/2023.findings-acl.67.

[5] G. Agrawal, T. Kumarage, Z. Alghamdi, H. Liu, Can knowledge graphs reduce hallucinations in llms? : A survey, 2024. URL: https://arxiv.org/abs/2311.07914. arXiv:2311.07914.

[6] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. d. Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, A. Zimmermann, Knowledge graphs, ACM Computing Surveys 54 (2021) 1–37. URL: https://dl.acm.org/doi/10.1145/3447772. doi:10.1145/3447772.

[7] T. R. Gruber, A translation approach to portable ontology specifications, Knowledge Acquisition 5 (1993) 199–220. URL: https://www.sciencedirect.com/science/article/pii/S1042814383710083. doi:https://doi.org/10.1006/knac.1993.1008.

[8] Neo4j, Inc., Neo4j graph database, 2024. URL: https://neo4j.com, version 5. Accessed 10 June 2025.

[9] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, X. Wu, Unifying large language models and knowledge graphs: A roadmap, IEEE Transactions on Knowledge and Data Engineering 36 (2024) 3580–3599. doi:10.1109/TKDE.2024.3352100.

[10] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020, pp. 9459–9474. URL: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

[11] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, J. Larson, From local to global: A graph rag approach to query-focused summarization, 2025. URL: https://arxiv.org/abs/2404.16130. arXiv:2404.16130.

[12] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, H. Wang, Retrieval-augmented generation for large language models: A survey, 2024. URL: https://arxiv.org/abs/2312.10997. arXiv:2312.10997.

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems 30 (NIPS 2017), 2017, pp. 5998–6008. URL: https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[14] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, in: Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 2020, pp. 1877–1901. URL: https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html.

[15] R. Diestel, Graph Theory, volume 173 of *Graduate Texts in Mathematics*, 5th ed., Springer, 2017. doi:10.1007/978-3-662-53622-3.

[16] Chase, Harrison, Langchain, 2023. URL: https://github.com/langchain-ai/langchain.

[17] A. Nadgeri, A. Bastos, K. Singh, I. O. Mulang', J. Hoffart, S. Shekarpour, V. Saraswat, KGPool: Dynamic knowledge graph context selection for relation extraction, in: C. Zong, F. Xia, W. Li, R. Navigli (Eds.), Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021, Association for Computational Linguistics, Online, 2021, pp. 535–548. URL: https://aclanthology.org/2021.findings-acl.48/. doi:10.18653/v1/2021.findings-acl.48.

[18] Z. Zheng, Y. Dong, S. Wang, H. Liu, Q. Wang, J. Li, Kg-cf: Knowledge graph completion with context filtering under the guidance of large language models, in: 2024 IEEE International Conference on Big Data (BigData), 2024, pp. 805–810. doi:10.1109/BigData62323.2024.10826107.

[19] C. Xu, M. Li, C. Yang, X. Jiang, L. Tang, Y. Qi, J. Guo, Context graph, 2024. URL: https://arxiv.org/abs/2406.11160. arXiv:2406.11160.

[20] L. Zhang, Z. Jiang, H. Chi, H. Chen, M. Elkoumy, F. Wang, Q. Wu, Z. Zhou, S. Pan, S. Wang, Y. Ma, Diagnosing and addressing pitfalls in kg-rag datasets: Toward more reliable benchmarking, 2025. URL: https://arxiv.org/abs/2505.23495. arXiv:2505.23495.

[21] T. Zhang*, V. Kishore*, F. Wu*, K. Q. Weinberger, Y. Artzi, Bertscore: Evaluating text generation with bert, in: International Conference on Learning Representations, 2020. URL: https://openreview.net/forum?id=SkeHuCVFDr.

[22] E. B. Wilson, Probable inference, the law of succession, and statistical inference, Journal of the American Statistical Association 22 (1927) 209–212. doi:10.1080/01621459.1927.10502953.

[23] Q. McNemar, Note on the sampling error of the difference between correlated proportions or percentages, Psychometrika 12 (1947) 153–157. doi:10.1007/BF02295996.

[24] J. Cohen, A coefficient of agreement for nominal scales, Educational and Psychological Measurement 20 (1960) 37–46. doi:10.1177/001316446002000104.